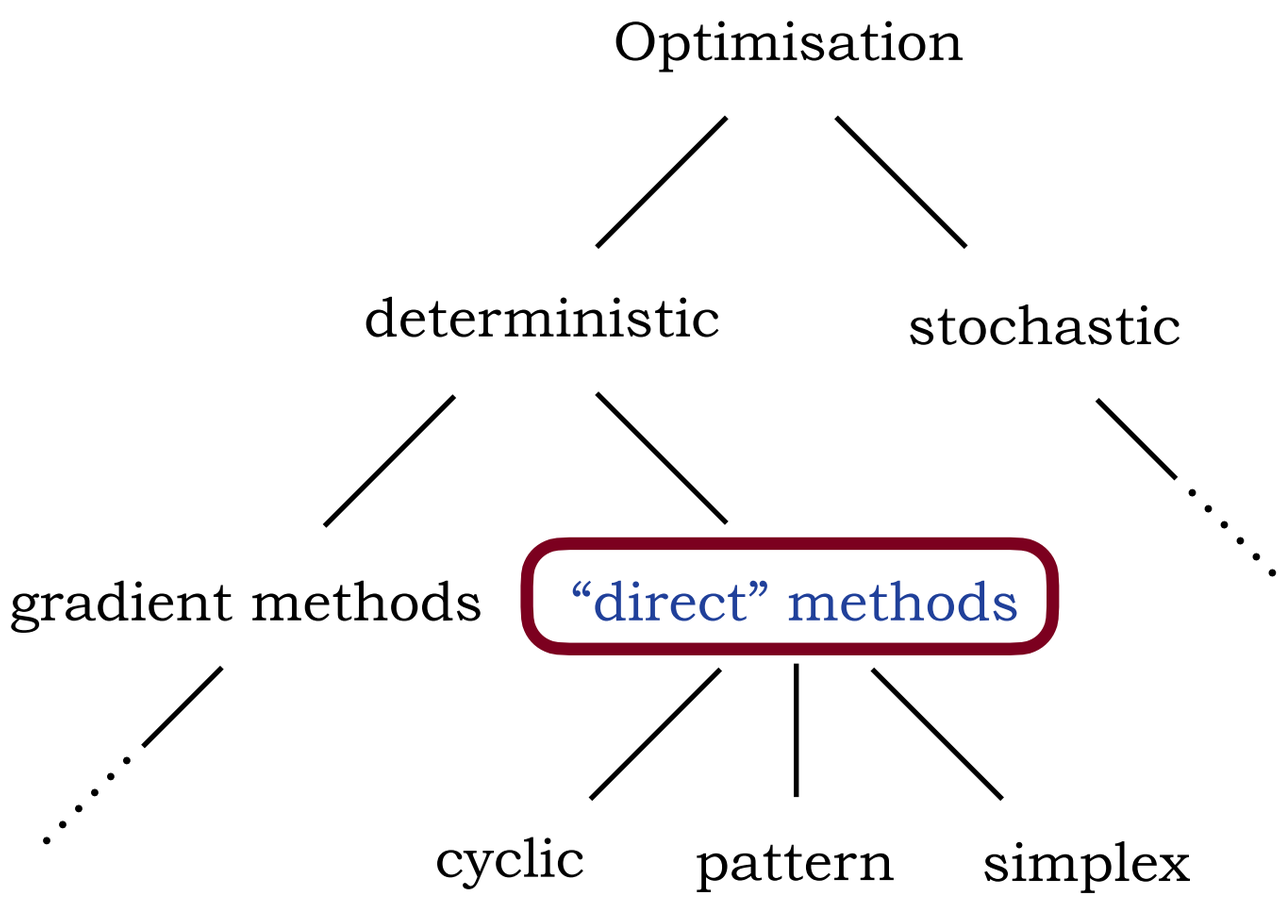
Direct Methods

Reference

Algorithms for Optimization
Mykel Kochenderfer, Tim Wheeler, MIT Press, 2019

© Siwen Luo, UWA

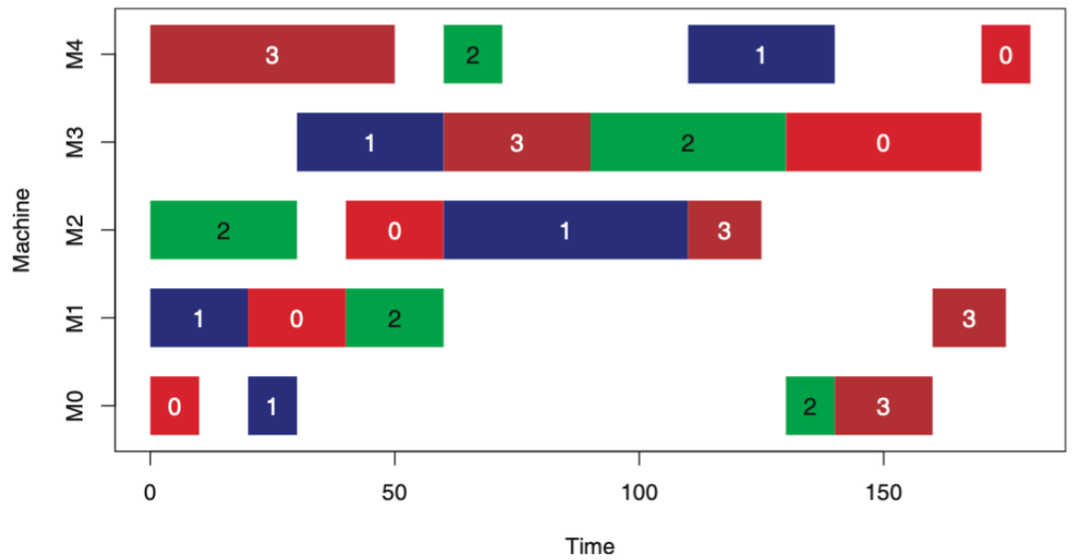
Optimisation Algorithms - Roadmap



Continuously differentiable → Not continuously differentiable

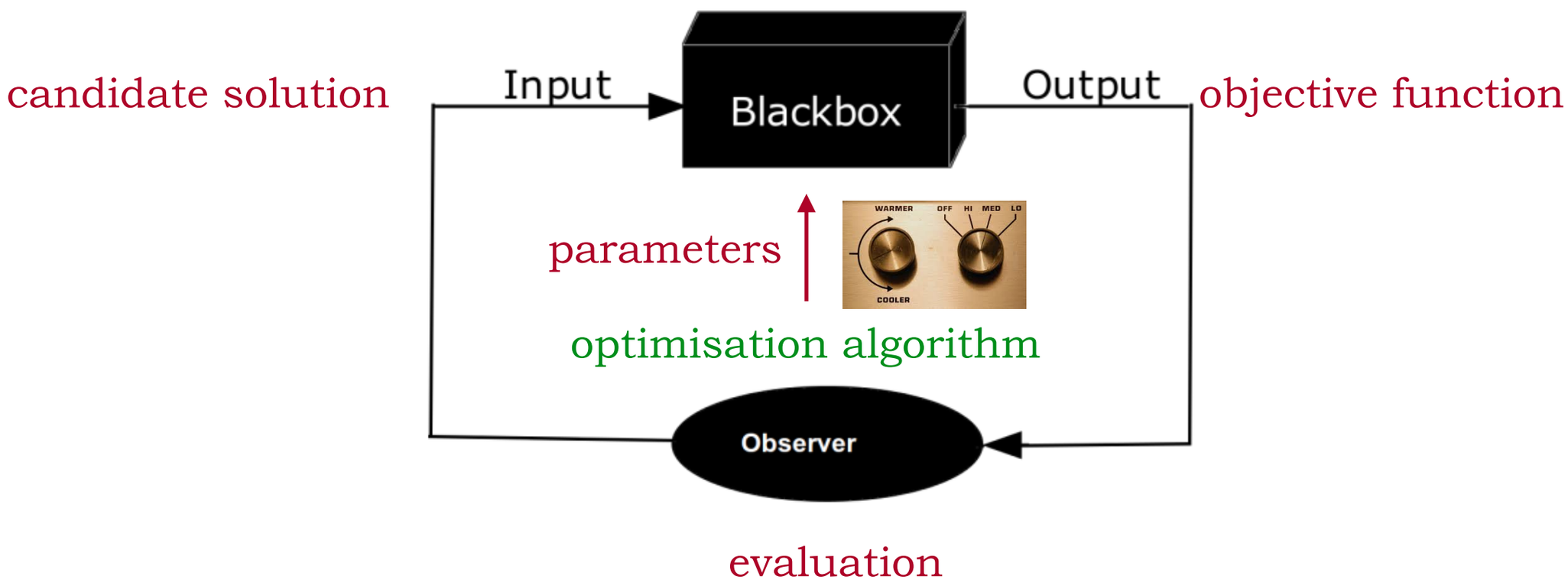
What are Direct Methods?

- Gradient methods assume
  - access to gradient (1st derivative) and in some cases higher derivatives
  - (not necessarily ability to evaluate function itself)
- In general
  - derivatives are not available
    - search space may be smooth, but very complex
    - search space may not be differentiable
    - search space may not be continuous
- Example
  - Gantt charts from the JSSP



What are Direct Methods?

- In general, derivatives are not available
  - worst case, search space may effectively be a “black box”
  - at the very least, we may be able to evaluate it
  - that is, a measure of how “good” our solution is
    - ➡ ***objective function*** (cost function, loss function, fitness,…)



What are Direct Methods?

- ***Direct methods*** rely solely on the objective function
  - seek an optimum by ***sampling***, however…
  - each time you evaluate a candidate solution, you're not just getting a number for the candidate
    - ➡ you ***learn*** a bit more about the space!
  - in fact, you might not care about the candidate at all - just using it to get *more information*
- What kind of information do we want?
  - we'll see there can be a lot - lots of room for ingenuity
  - but the most obvious is information about, or a local approximation to, the slope/derivative!
- Let's look at some algorithms…

Cyclic Coordinate Search

Also known as ***coordinate descent*** or ***taxicab search***

Assume the objective function is

$$f(\vec{x}) = f(x_1, x_2, \ldots, x_n)$$

Starting from initial value $\vec{x}^0$

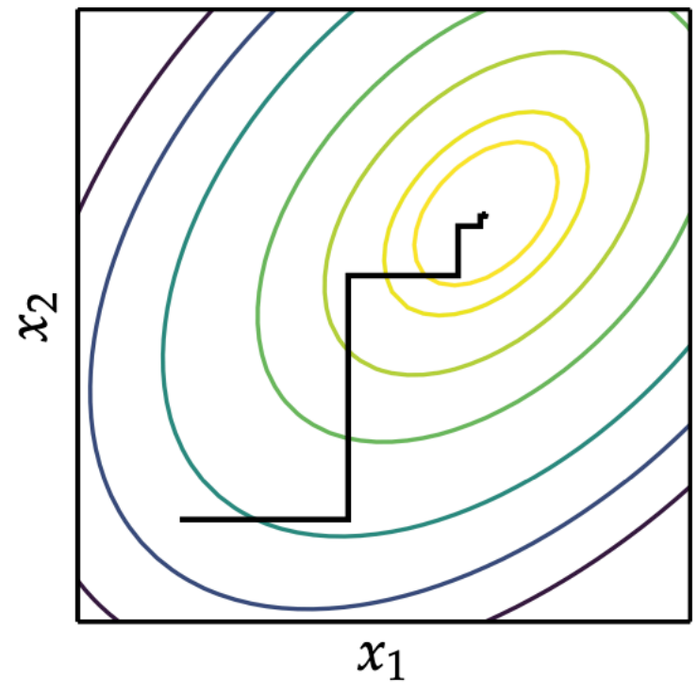
$$\vec{x}^1 = \arg\min_{x_1} \; f(x_1, x_2^0, x_3^0, \ldots, x_n^0)$$

$$\vec{x}^2 = \arg\min_{x_2} \; f(x_1^1, x_2, x_3^1, \ldots, x_n^1)$$

etc…

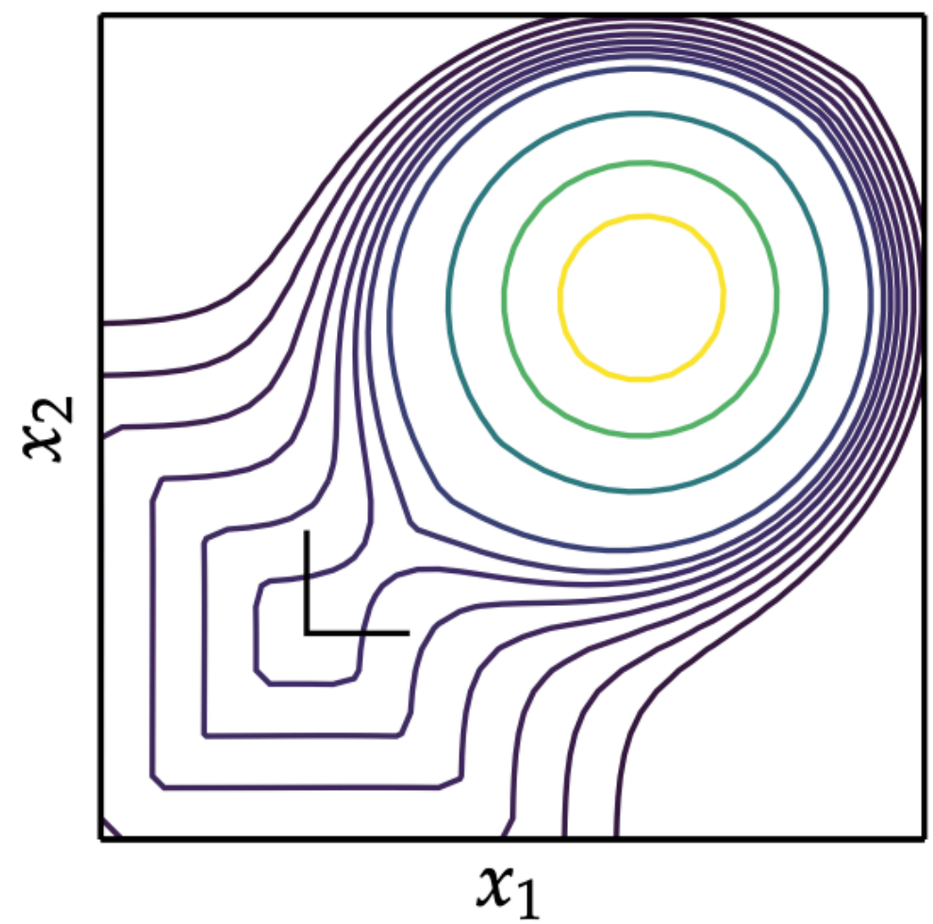
- series of *line search* optimisations

- After full cycle, start again from x_1…



Cyclic Coordinate Search

- Will either improve or stay the same with each iteration
- "Converged" after full cycle with no (significant) improvement
- Can fail to find local optimum…



Cyclic Coordinate Search

```
function cyclic_coordinate_descent(f, x, ϵ)
    Δ, n = Inf, length(x)
    while abs(Δ) > ϵ
        x′ = copy(x)
        for i in 1 : n
            d = basis(i, n)
            x = line_search(f, x, d)
        end
        Δ = norm(x - x′)
    end
    return x
end
```

basis functions

direction

norms again..

$|\Delta| \leq \epsilon$ is the stopping criterion

ϵ may be called the “tolerance”

norm is the “distance" between the points - eg. recall the *Euclidian* or L^2 *norm*:

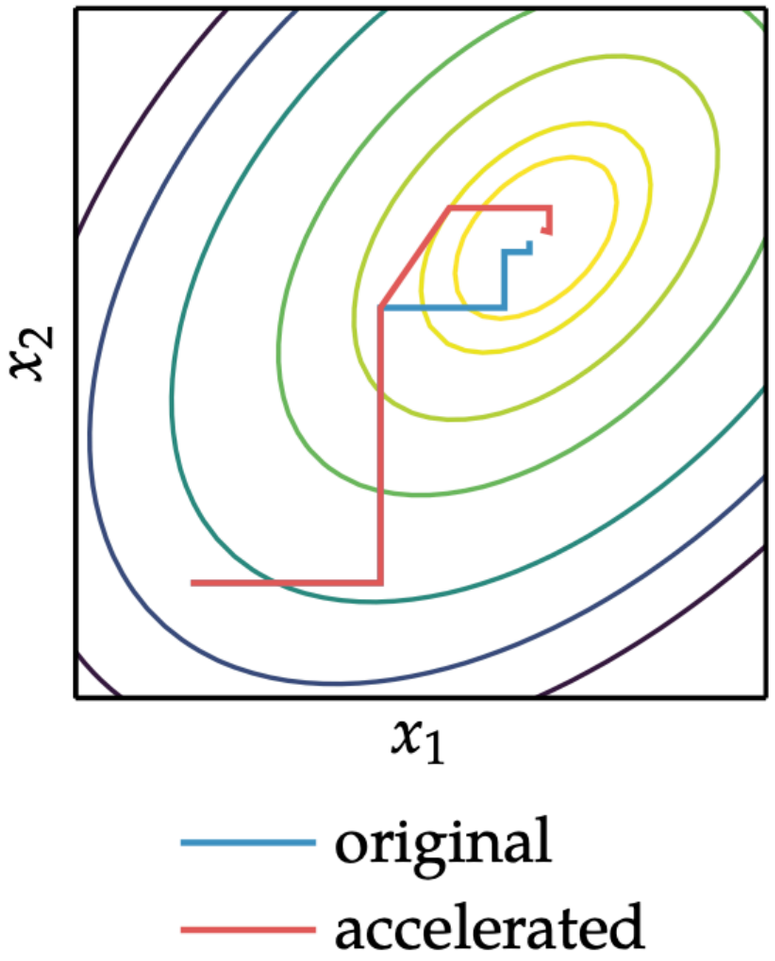
$$\|\vec{x}\|_2 = \sqrt{x_1^2 + \cdots x_n^2}$$

Cyclic Coordinate Search with "Acceleration Step"

- After one cycle, assume that the *net* progress is "pointing in the right direction"
  - note - *not coordinate (basis) direction*
- That direction after the first n step cycle is given by the vector difference

$$\vec{x}^n - \vec{x}^0$$

  - similarly for $2n$, etc…
- Take one additional step in the calculated direction
- Faster to traverse diagonal "valleys/ ridges"



Cyclic Coordinate Search with "Acceleration Step"

```
function cyclic_coordinate_descent_with_acceleration_step(f, x, ϵ)
    Δ, n = Inf, length(x)
    while abs(Δ) > ϵ
        x′ = copy(x)
        for i in 1 : n
            d = basis(i, n)
            x = line_search(f, x, d)
        end
        x = line_search(f, x, x - x′) # acceleration step
        Δ = norm(x - x′)
    end
    return x
end
```

Powell's Method

- Extends on this idea by maintaining a *queue of search directions*

- Notice that in CCS

$$\vec{x}^1 = \arg\min_{x_1} f(x_1, x_2^0, x_3^0, \ldots, x_n^0)$$

conducts a line search from $\vec{x}^0$ in the direction of the x_1 axis.

That is, it searches in the direction of the *basis vector*

(a.k.a. *standard unit vector*)

$$\vec{e}_1 = [1,0,\ldots,0]$$

So CCS searches in the direction of each of the n basis vectors in turn, then repeats.

Powell's Method

Powell's Method generalises this by allowing the search directions to be adapted (in non-basis directions) during the search:

1. Assume the "unit direction" vectors are stored in a queue

$$[\vec{u}_1, \vec{u}_2, \ldots, \vec{u}_n]$$

where initially

$$[\vec{u}_1, \vec{u}_2, \ldots, \vec{u}_n] = [\vec{e}_1, \vec{e}_2, \ldots, \vec{e}_n]$$

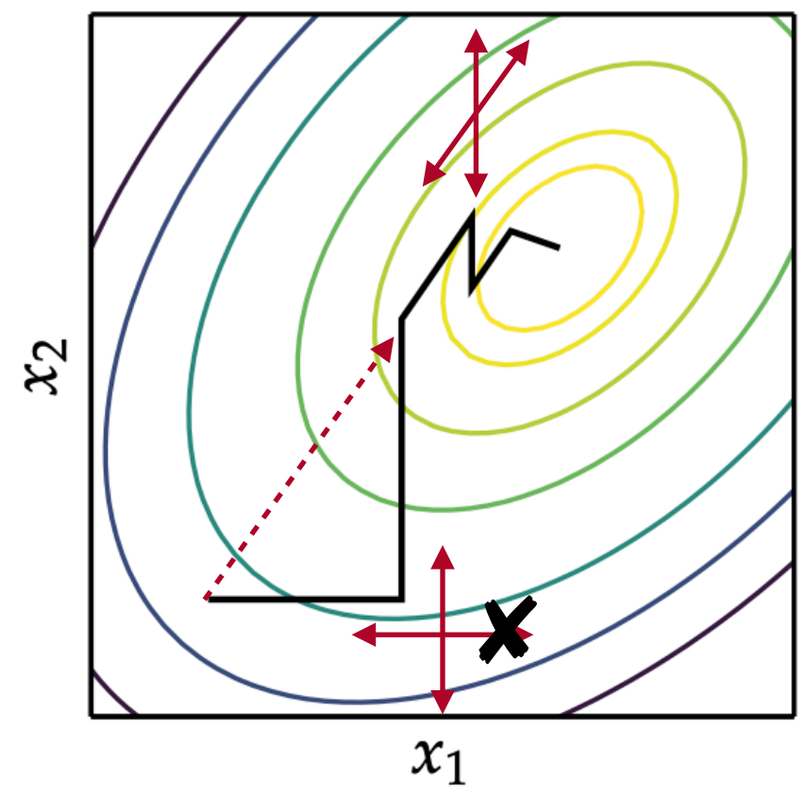
2. Complete a cycle of n line searches in the directions $\vec{u}_1$ to $\vec{u}_n$ (as per CCS)

3. Calculate

$$\vec{u}_{n+1} = \vec{x}^n - \vec{x}^0$$

Powell's Method

4. Calculate a new value of $\vec{x}$ using $\vec{u}_{n+1}$ (cf acceleration step in CCS)
5. Dequeue $\vec{u}_1$ from the front of the queue
6. Enqueue $\vec{u}_{n+1}$ to the back of the queue
7. Repeat from 2 using the new queue of directions, until convergence

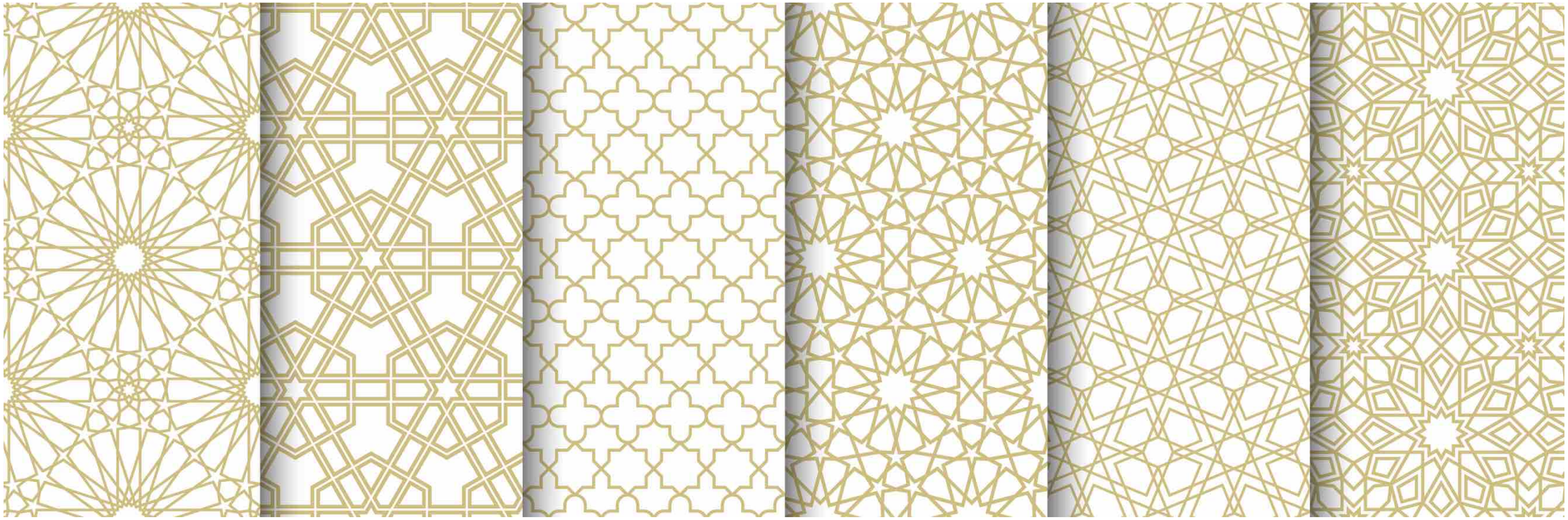


- Q: The method can degenerate so it no longer has access to the full space - how?
- For those familiar with vector spaces, what happens each time one vector becomes *linearly dependent* on others?
- What is an advantage of the original basis vectors? (in terms of *span*)

Powell's Method

```
function powell(f, x, ϵ)
    n = length(x)
    U = [basis(i,n) for i in 1 : n]
    Δ = Inf
    while Δ > ϵ
        x′ = x
        for i in 1 : n
            d = U[i]
            x′ = line_search(f, x′, d)
        end
        for i in 1 : n-1
            U[i] = U[i+1]
        end
        U[n] = d = x′ - x
        x′ = line_search(f, x′, d)
        Δ = norm(x′ - x)
        x = x′
    end
    return x
end
```

Pattern Search Methods



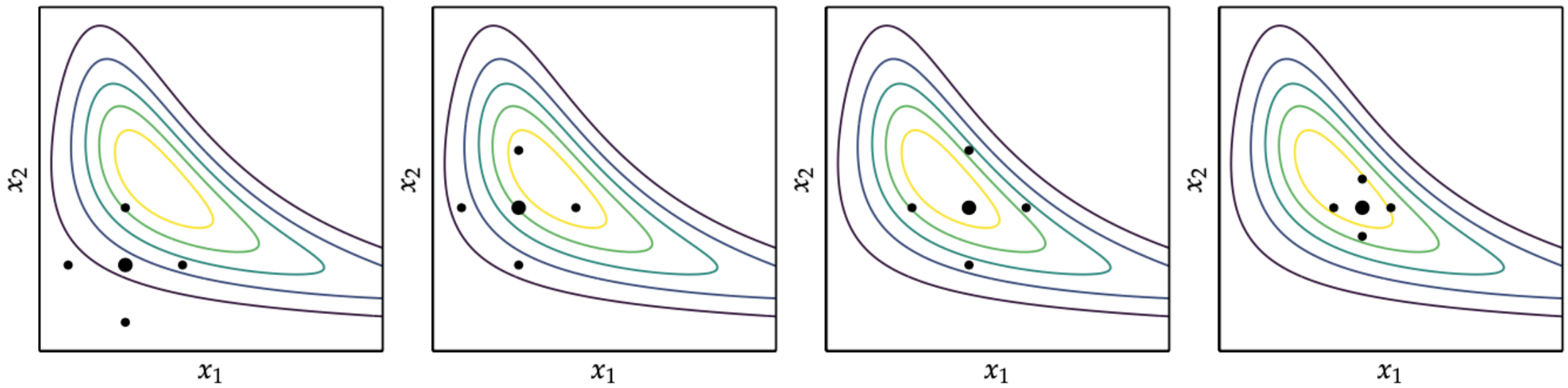
Hooke-Jeeves Method

- CCS with acceleration and Powell's method use some information from previous samples to *inform* a - hopefully - better search direction
  - The ***heuristic*** is something like *"the direction I've come is likely to be a promising direction to keep going…"*
- *Hooke-Jeeves* instead uses more samples to try to find the best direction for each step
  - evaluates

    $$f(\vec{x}) \text{ and } f(\vec{x} \pm \alpha \vec{e}_i)$$

    - for each dimension i and a step size α around "anchor point" $\vec{x}$
    - directly sampling slope (*cf.* derivative)
  - if at least one sample is an improvement, take the best direction and repeat
  - if none are improvements, reduce step size α and repeat (ie. "shrink", why?)

Hooke-Jeeves Method



- H-J requires *2n evaluations per step* for *n*-dimensional problem
  - we didn't specify how many evals in line search for CCS, Powell
  - how might we compare them?

Hooke-Jeeves Method

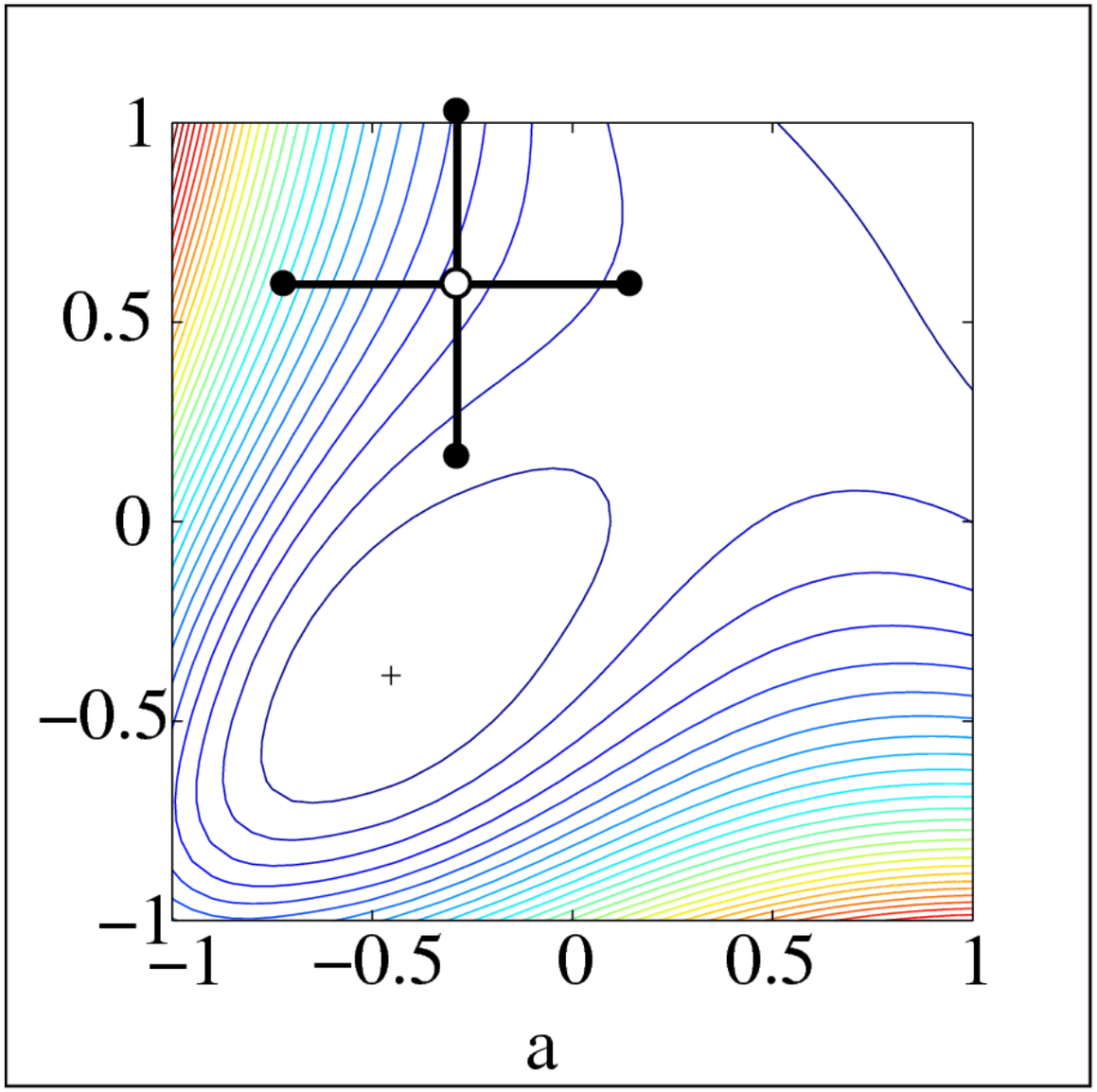
```
function hooke_jeeves(f, x, α, ϵ, γ=0.5)
    y, n = f(x), length(x)
    while α > ϵ
        improved = false
        x_best, y_best = x, y
        for i in 1 : n
            for sgn in (-1,1)
                x′ = x + sgn*α*basis(i, n)
                y′ = f(x′)
                if y′ < y_best
                    x_best, y_best, improved = x′, y′, true
                end
            end
        end
        x, y = x_best, y_best

        if !improved
            α *= γ
        end
    end
    return x
end
```

α - step size

ϵ - tolerance

γ - step decay

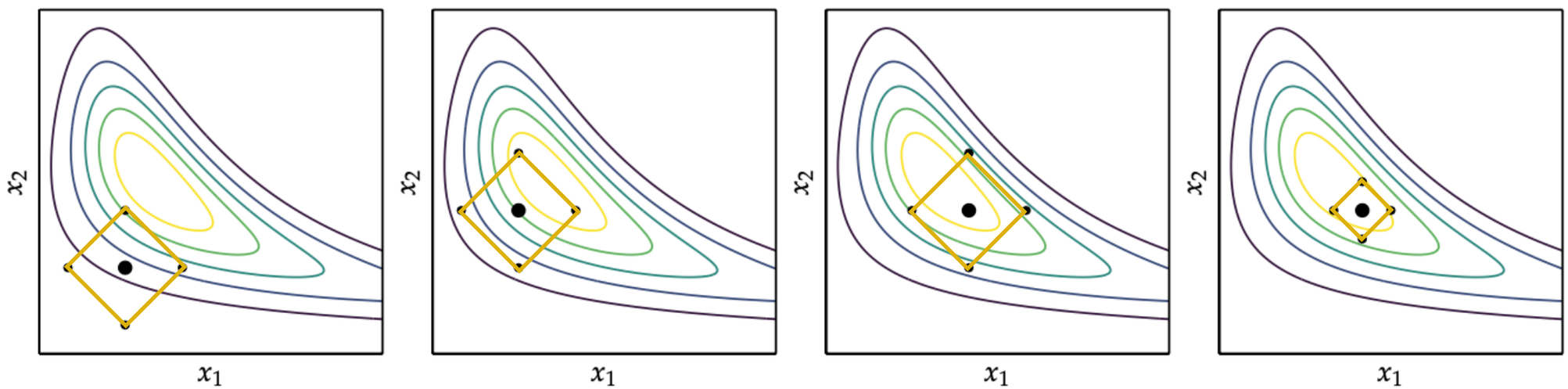
Hooke-Jeeves Method



Author: Guillaume Jacquenot, Wikimedia Commons
Function: Broyden Function

Generalised Pattern Search

- H-J is an example of a ***pattern search***
  - can also be thought of "sliding" and shrinking an *n*-cube through space…

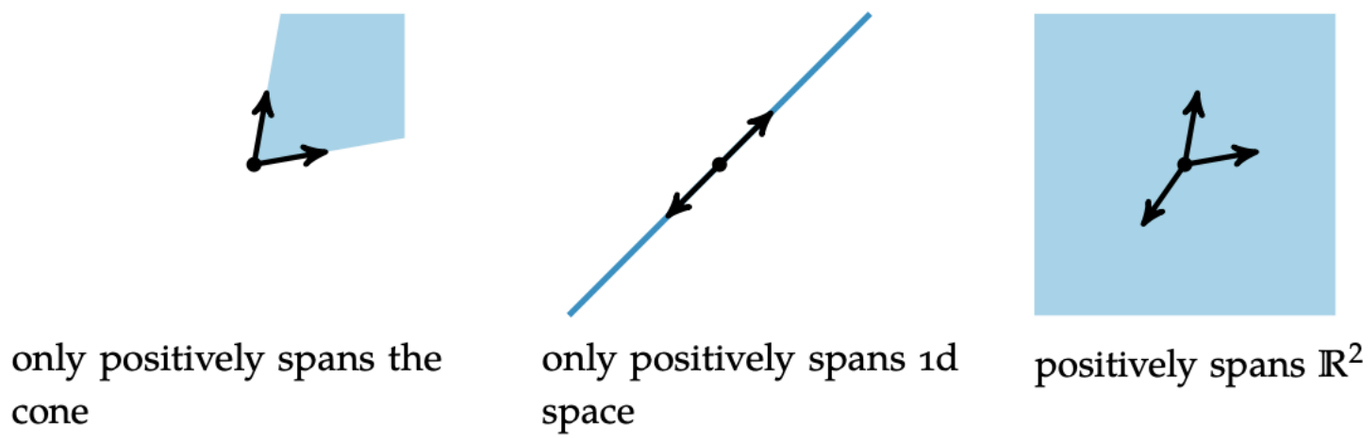


- Restricted to coordinate directions
- Can *generalise* using a set of directions D (*cf.* Powell)
- A ***pattern*** is defined by

$$P = \{\vec{x} + \alpha\vec{d} \mid d \in D\}$$

Generalised Pattern Search

- To ensure convergence to a local minimum, D must be a ***positive spanning*** set
  - ➡ can construct any point in $\mathbb{R}^n$ as a nonnegative linear combination of the directions in D



  - ➡ ensures at least one of the directions in D is a *descent* direction from a location with a non-zero gradient

Q: While H-J uses $2n$ directions, GPS can use as few as $n + 1$ - *why?*

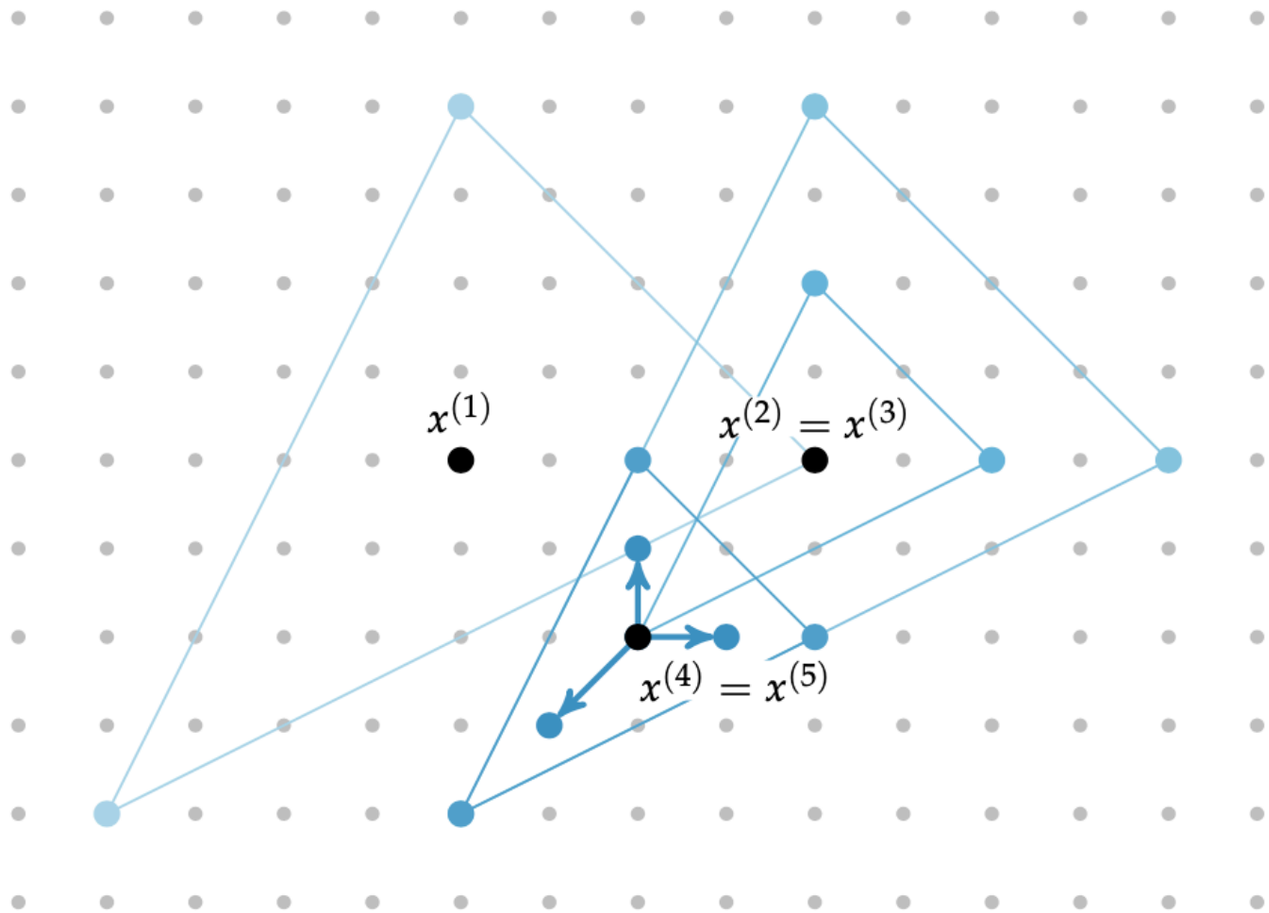
Generalised Pattern Search

```
function generalized_pattern_search(f, x, α, D, ϵ, γ=0.5)
    y, n = f(x), length(x)
    while α > ϵ
        improved = false
        for (i,d) in enumerate(D)
            x′ = x + α*d
            y′ = f(x′)
            if y′ < y
                x, y, improved = x′, y′, true
                D = pushfirst!(deleteat!(D, i), d)
                break
            end
        end
        if !improved
            α *= γ
        end
    end
    return x
end
```

Options

- *opportunistic* - first improved value is accepted (trade-off)
- *dynamic ordering* - direction that leads to improvement is pushed to beginning of queue

Generalised Pattern Search



- Note directions don't change
  - ➡ can be thought of as sliding and shrinking an *n*-dimensional *pattern* on an *n*-dimensional lattice

Q: What is the minimal pattern in 2-d? In 3-d?

Nelder-Mead Simplex Method

Remember the Peter's Freeza?

Only optimisation method based on an ice cream?

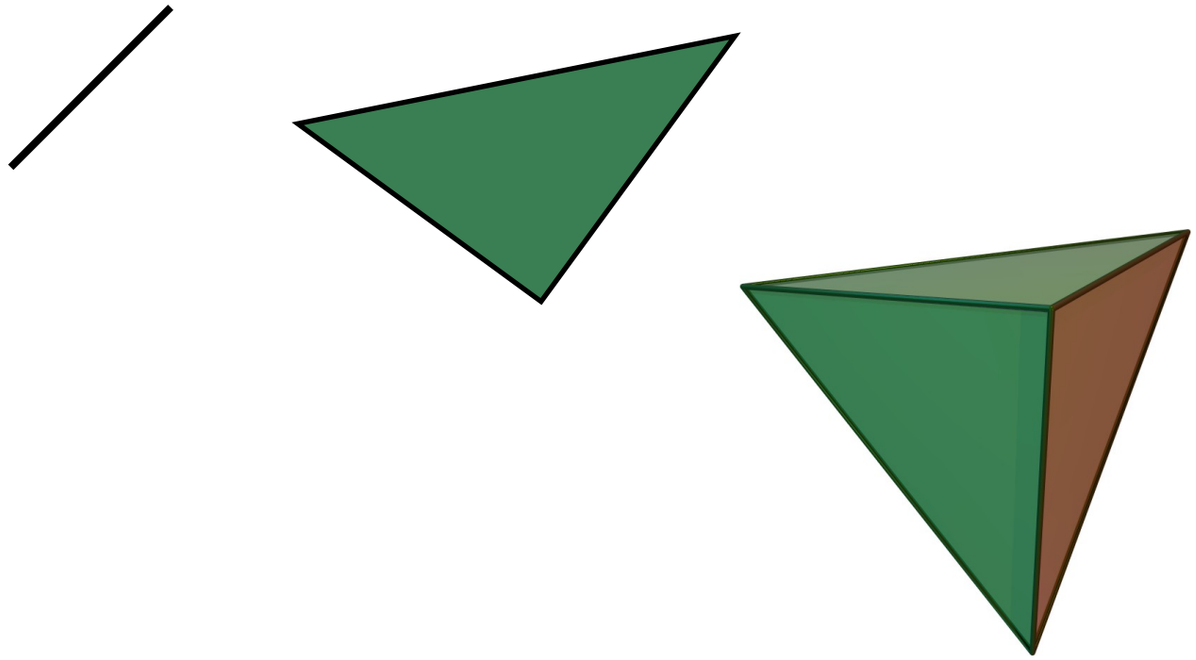
Here's how it works…

Nelder-Mead Simplex Method

Mount Nameless, Tom Price

Nelder-Mead Simplex

- Technically, the Peter's Freeza is a *tetrahedron.*
- A ***simplex*** is a generalisation of the tetrahedron in *n*-dimensional space.
  - 1-dimension - line segment
  - 2-dimensional - triangle
  - 3-dimensional - tetrahedron
  - 4-dimensional - 🤯
  - 5-dimensional…



- Called a *simplex* as its the simplest polytope (flat-faced object) in any given space.

Simplex Method

The simplex method (Nelder & Mead, 1965) - not to be confused with the simplex method used in linear programming.

- Set of rules determine how the pattern moves through the space ("rolls down the hill")
  - based on evaluations at vertices
  - unlike H-J (which only shrinks) it changes shape as it moves
  - like H-J shrinks as it converges on an optimum

- Let $\vec{x}_1, \ldots, \vec{x}_{n+1}$ be the vertices of a simplex
- Let $\vec{x}_h$, $\vec{x}_s$, $\vec{x}_l$ be the vertices with the *highest*, *second highest* and *lowest* objective function value
- Let $\bar{\vec{x}}$ be the mean of all points except $\vec{x}_h$, which we'll call the *centroid*

Simplex Operations

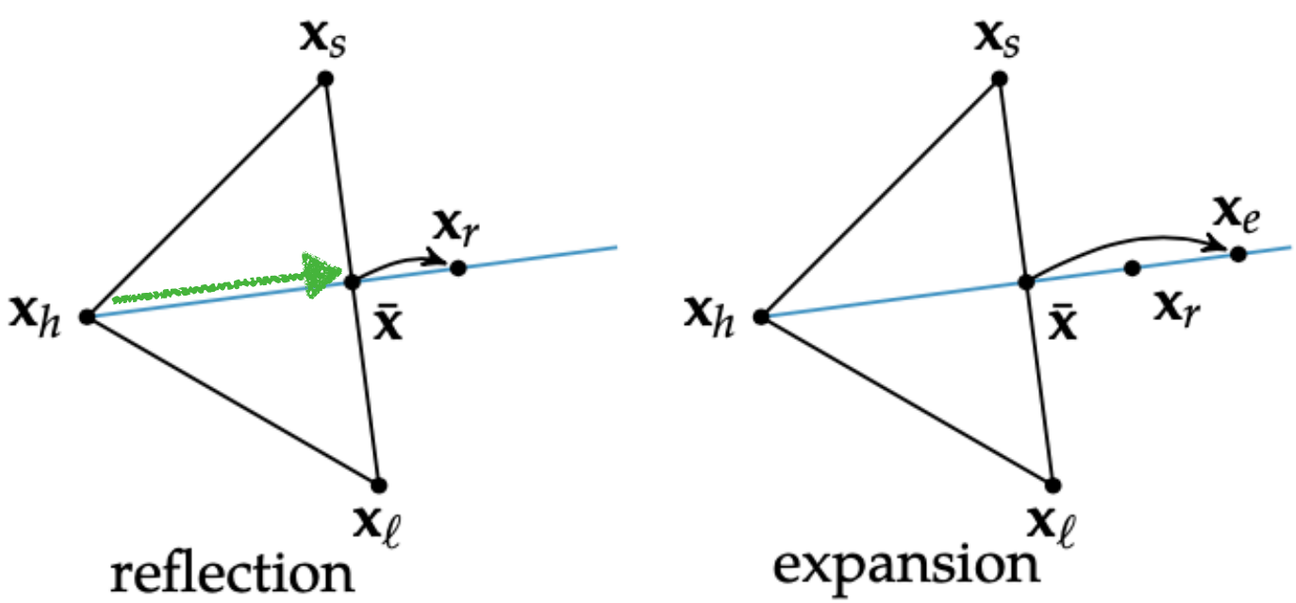
Reflection

$\vec{x}_h \rightarrow \vec{x}_r$ where

$\vec{x}_r = \vec{\bar{x}} + \alpha(\vec{\bar{x}} - \vec{x}_h), \quad \alpha > 0$ $\qquad$ α is typically set to 1

Expansion

$\vec{x}_e = \vec{\bar{x}} + \beta(\vec{x}_r - \vec{\bar{x}}), \quad \beta > \max(1,\alpha)$ $\qquad$ β is typically set to 2



Simplex Operations

Contraction

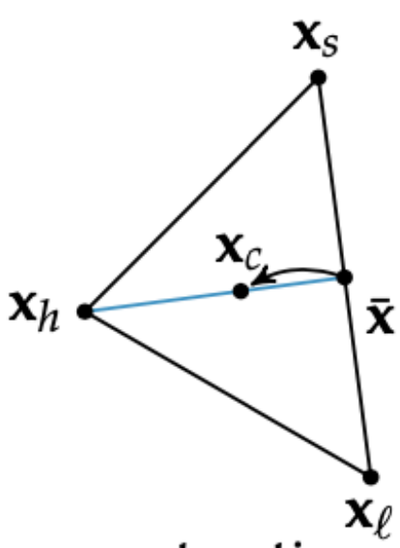
$$\vec{x}_c = \vec{\bar{x}} + \gamma(\vec{x}_h - \vec{\bar{x}}), \quad \gamma \in (0,1)$$

γ is typically set to 0.5

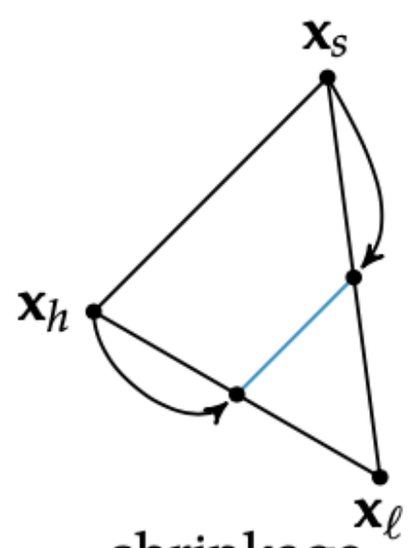
Shrinkage

$$\vec{x}_i = \vec{x}_l + \sigma(\vec{x}_i - \vec{x}_l), \quad \forall i \neq l, \quad \sigma \in (0,1)$$

σ is typically set to 0.5



contraction

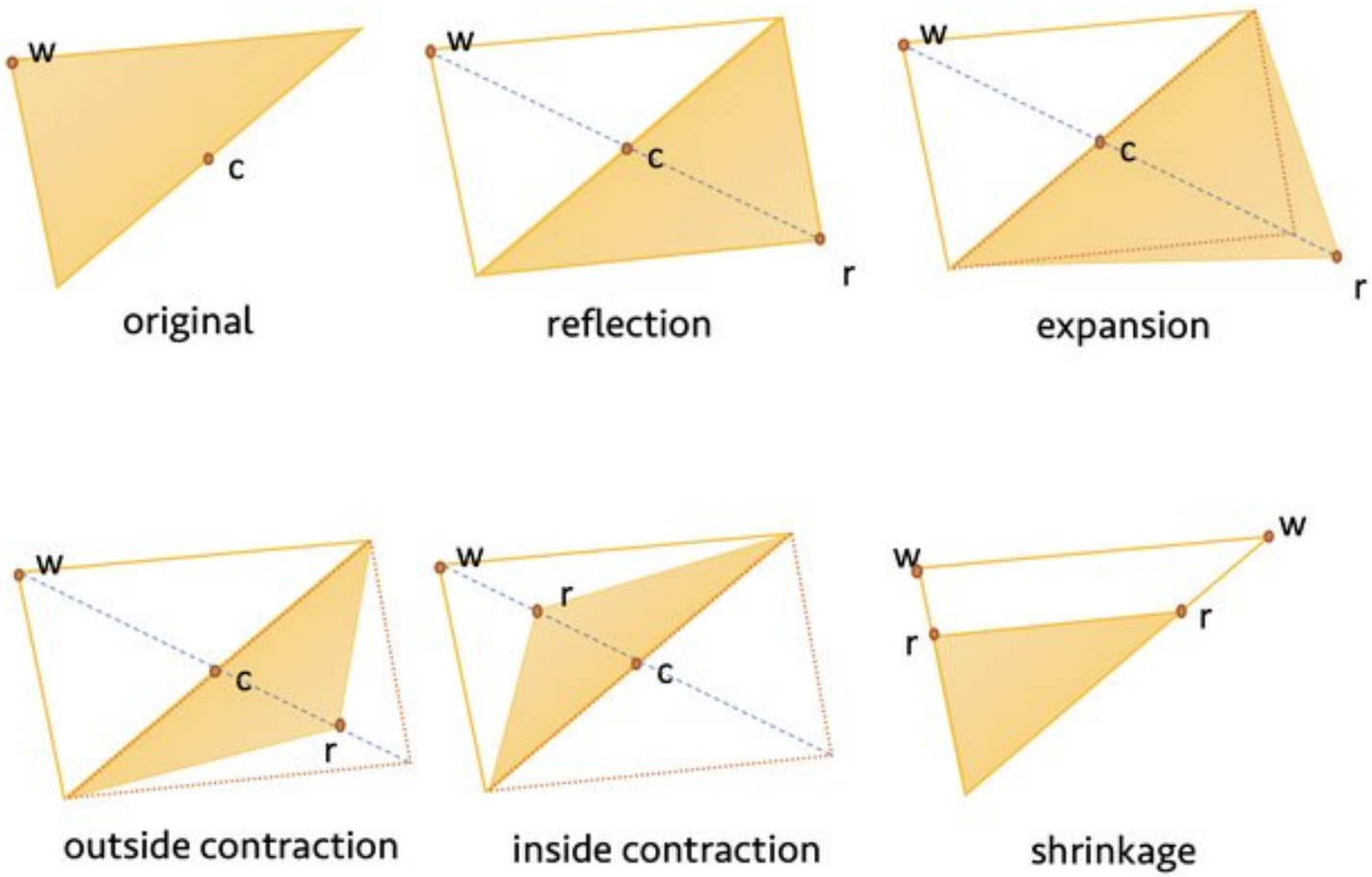


shrinkage

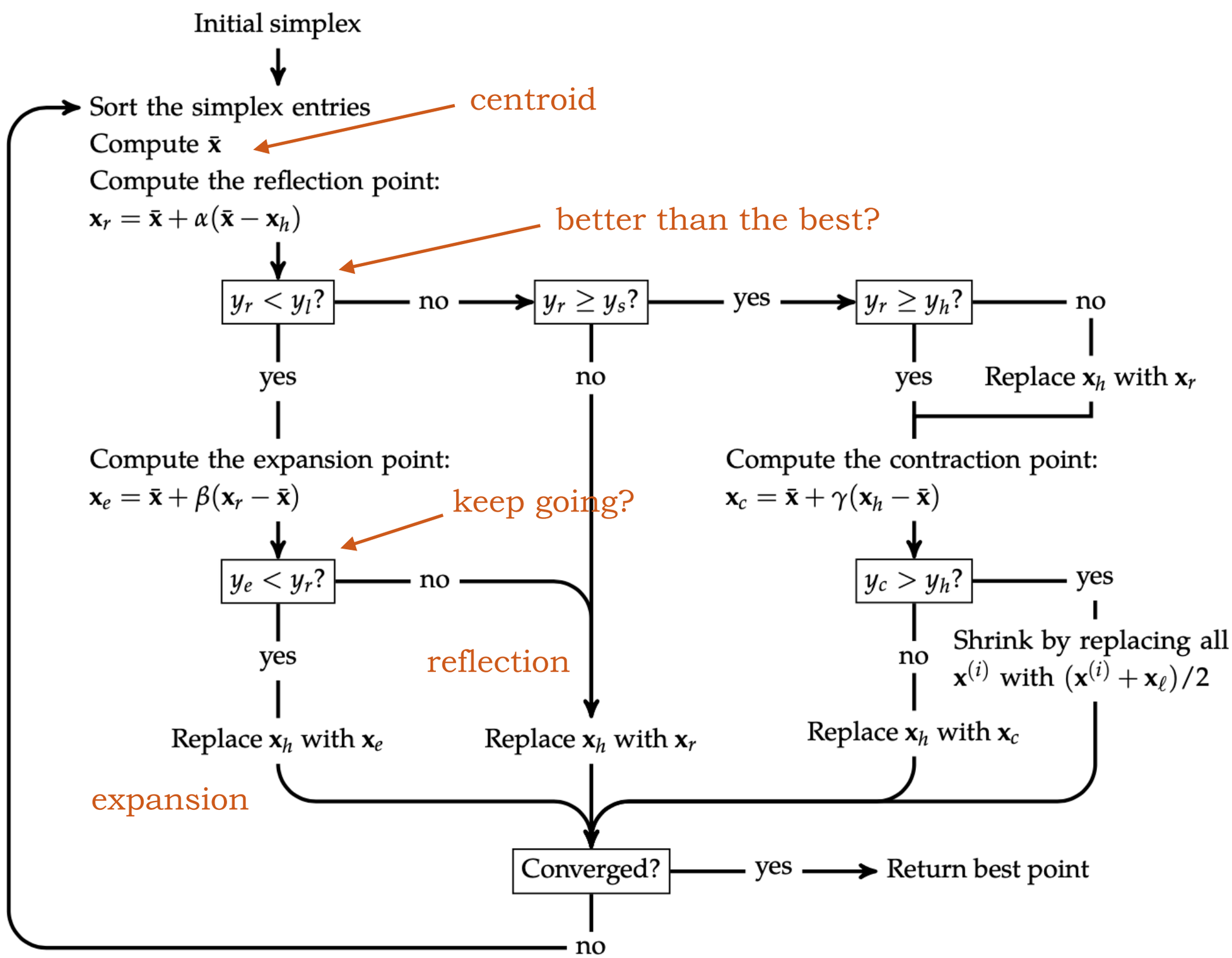
Simplex Operations

Some formulations also allow for *outside contraction*

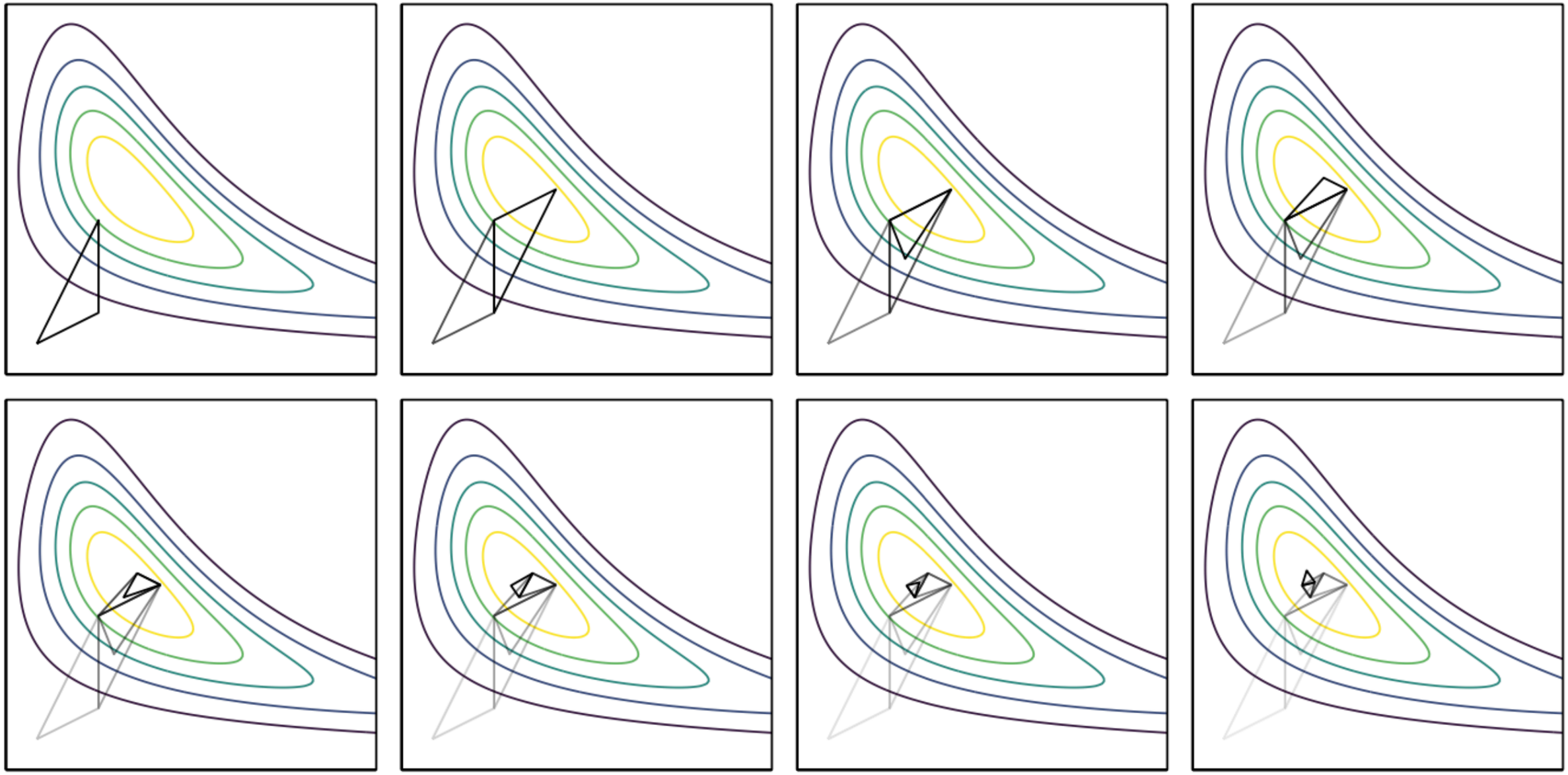
- $\vec{x}_c = \vec{\bar{x}} + \gamma(\vec{\bar{x}} - \vec{x}_h), \quad \gamma \in (0,1)$ γ is typically set to 0.5



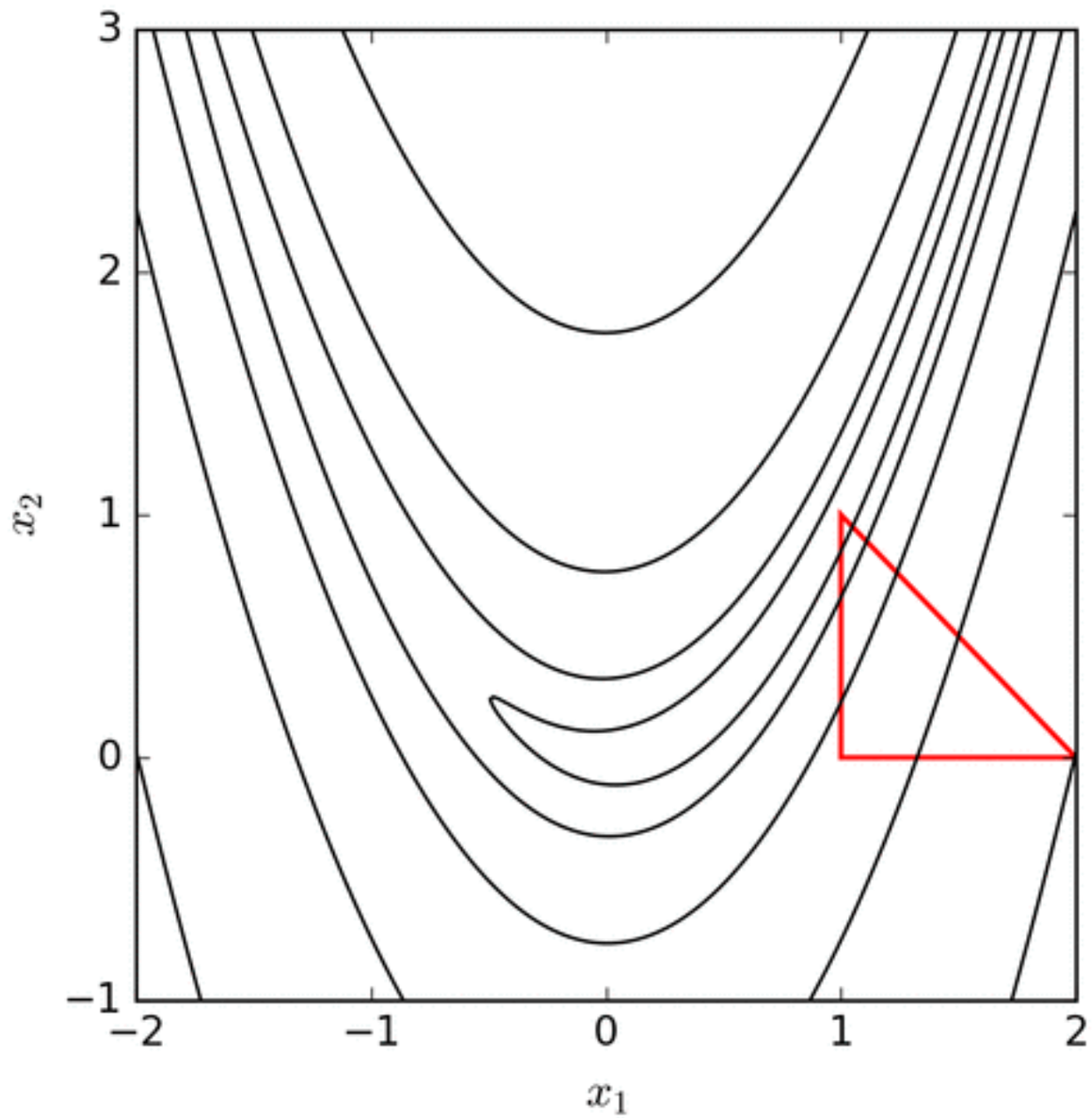
Flowchart for N-M Simplex

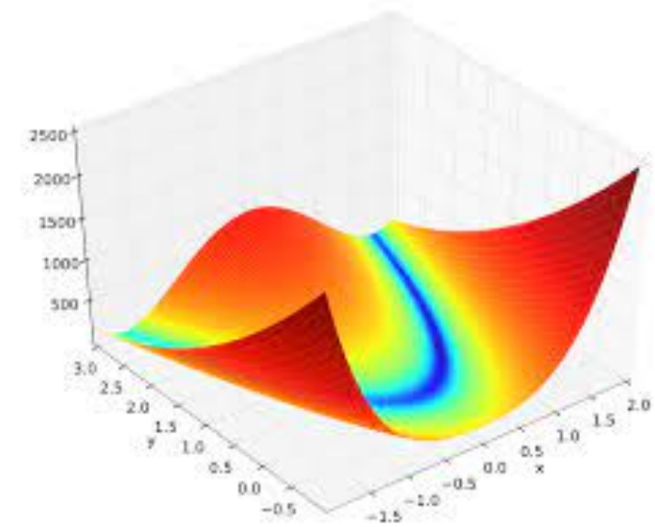


Simplex Algorithm

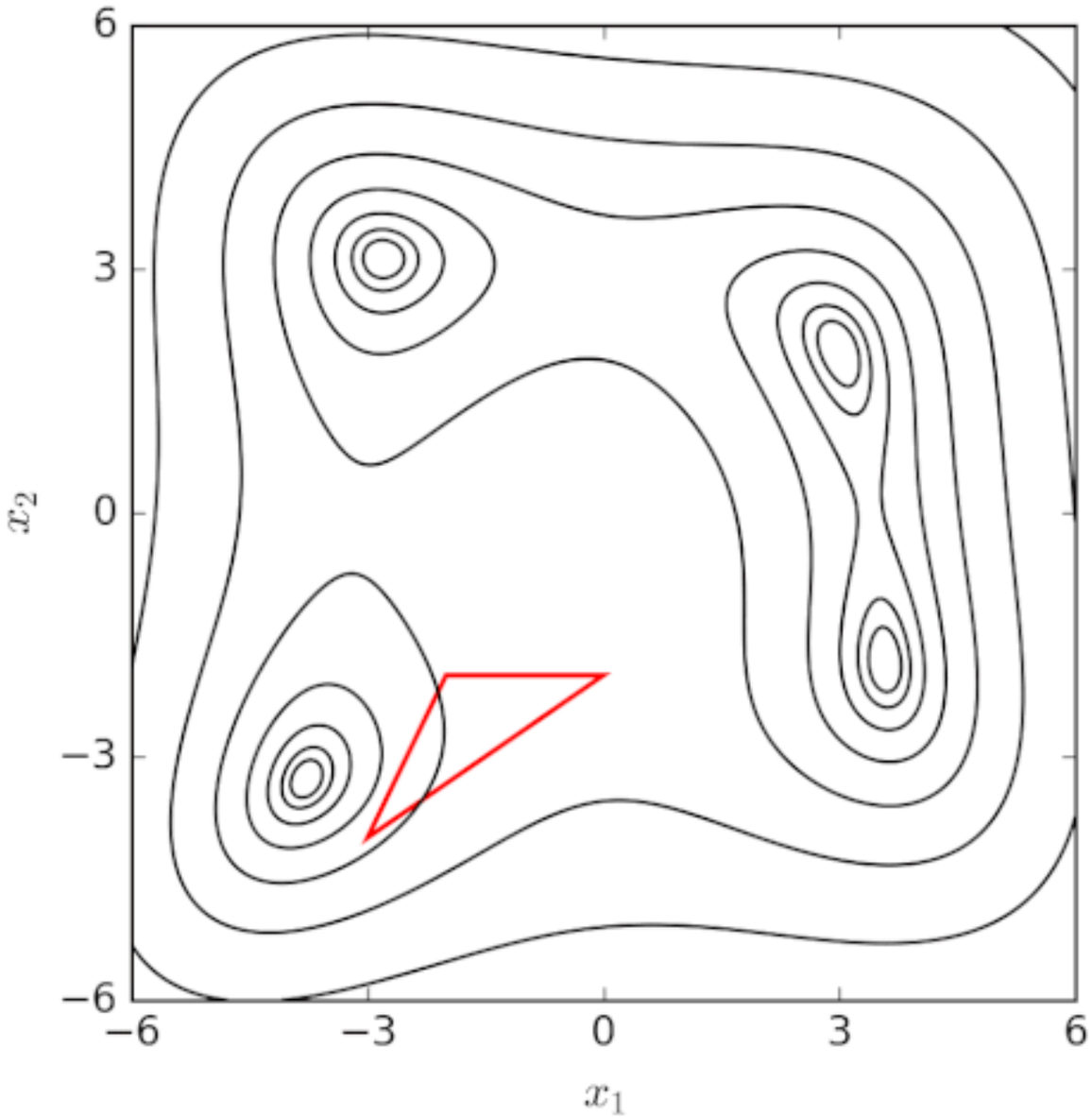


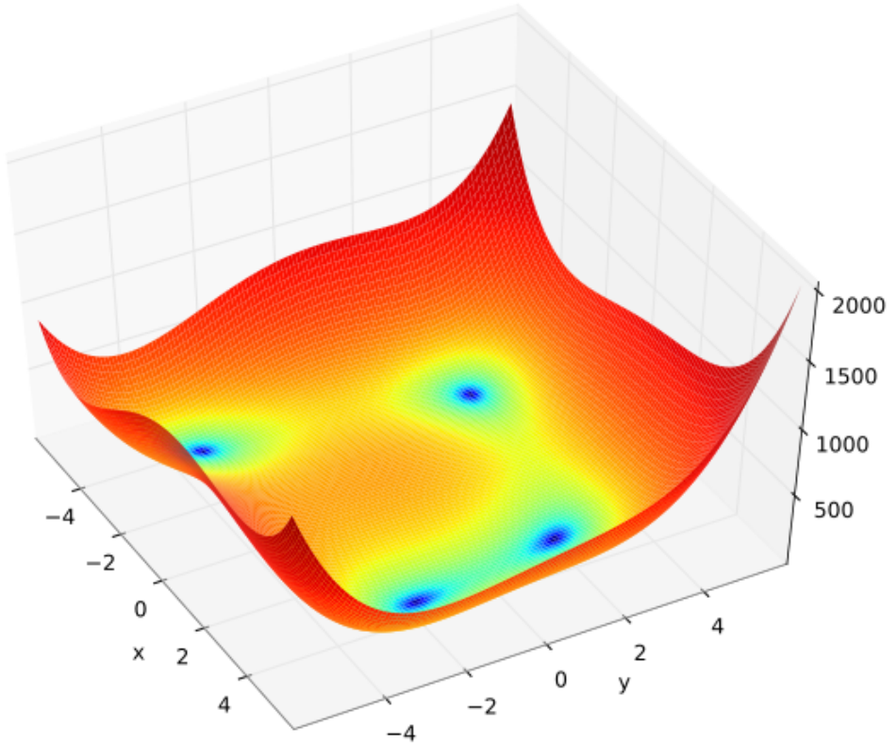
Example - Rosenbrock's Function





Example - Himmelblau's Function





Simplex Convergence

- We have seen stopping criteria including
  - step size $< \epsilon$
  - improvement $< \epsilon$
- For simplex have other alternatives
  - simplex is “sufficiently small” (similar to step size)
  - vertices are “sufficiently close”
    - ➡ we say that the points have ***converged***
- We can be craftier still…

Simplex Convergence

- Better still, use our evaluations to tell us something about the *curvature* of the "surface"
  - *cf.* second derivative in gradient descent!
- Use the **variance** (or std variation) of the vertex values as a proxy for curvature!
  - high for simplex in highly curved (or volatile) region
    - more gains to make
  - low for simplex in largely flat region
    - perhaps little gain for further compute
- Criterion

$$\sqrt{\frac{1}{n+1}\sum_{1}^{n+1}(y_i-\bar{y})^2} \;<\; \epsilon, \quad \text{where } y_i = f(\overrightarrow{x_i}) \text{ and } \bar{y} \text{ is the mean}$$

Simplex Algorithm

```
function nelder_mead(f, S, ϵ; α=1.0, β=2.0, γ=0.5)
    Δ, y_arr = Inf, f.(S)
    while Δ > ϵ
        p = sortperm(y_arr) # sort lowest to highest
        S, y_arr = S[p], y_arr[p]
        xl, yl = S[1], y_arr[1] # lowest
        xh, yh = S[end], y_arr[end] # highest
        xs, ys = S[end-1], y_arr[end-1] # second-highest
        xm = mean(S[1:end-1]) # centroid
        xr = xm + α*(xm - xh) # reflection point
        yr = f(xr)

        if yr < yl
            xe = xm + β*(xr-xm) # expansion point
            ye = f(xe)
            S[end],y_arr[end] = ye < yr ? (xe, ye) : (xr, yr)
        elseif yr ≥ ys
            if yr < yh
                xh, yh, S[end], y_arr[end] = xr, yr, xr, yr
            end
            xc = xm + γ*(xh - xm) # contraction point
            yc = f(xc)
            if yc > yh
                for i in 2 : length(y_arr)
                    S[i] = (S[i] + xl)/2
                    y_arr[i] = f(S[i])
                end
            else
                S[end], y_arr[end] = xc, yc
            end
        else
            S[end], y_arr[end] = xr, yr
        end

        Δ = std(y_arr, corrected=false)
    end
    return S[argmin(y_arr)]
end
```

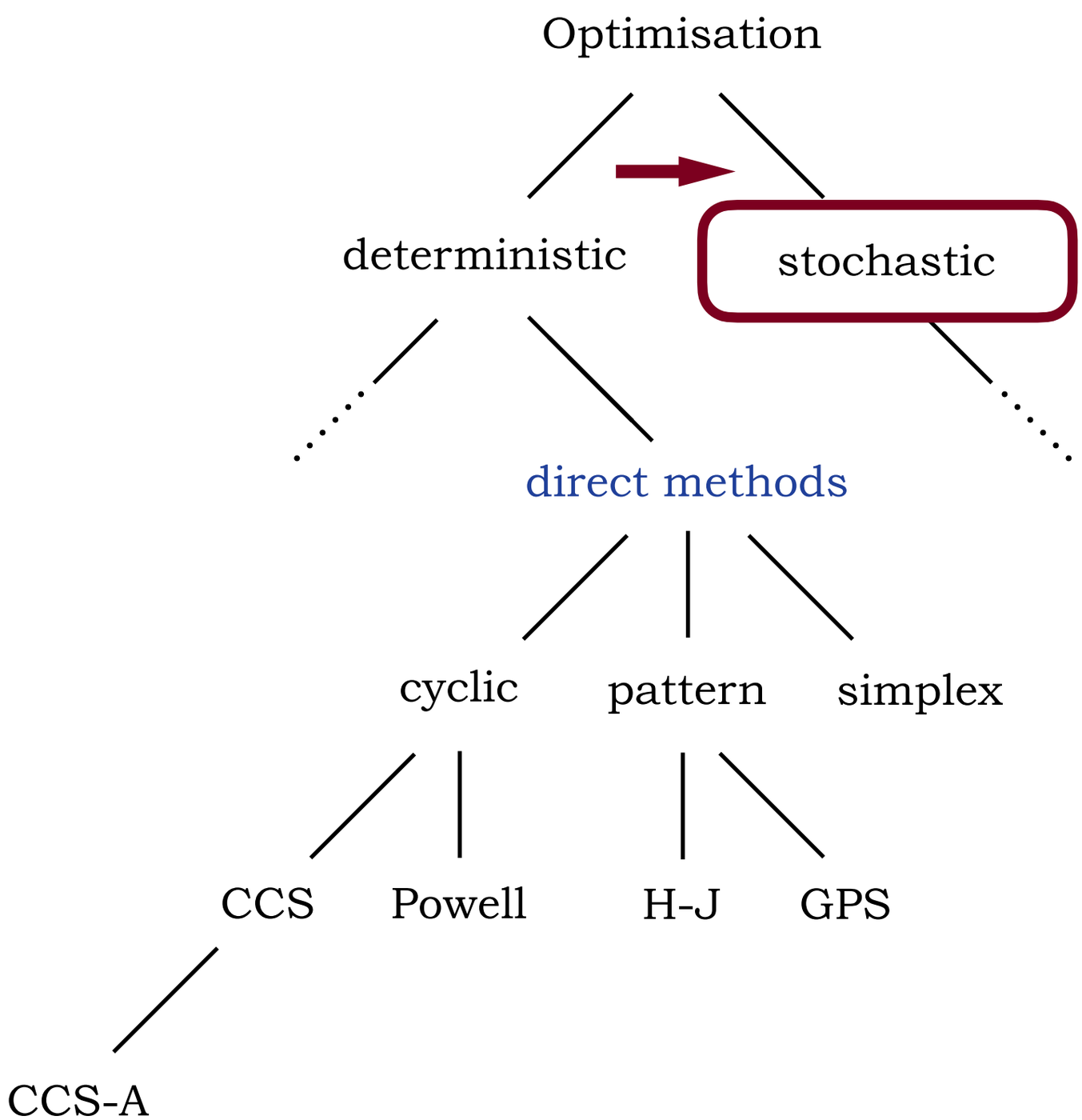
(See Kochenderfer Sec. 7.5 if too small to read)

“Collective Intelligence”

- Notice that with Simplex we are maintaining not just a single (best) solution - as we’ve done in the past
- Instead we are maintaining a small ***population*** of candidate solutions
- No one candidate can drive the algorithm on its own
  - *combined* information from the whole group is used to make each decision

➡ decisions made on the basis of the **relationship** between candidates!

Determinism

- Aside from choosing a starting point, the algorithms so far are ***deterministic***
  - ➡ the algorithm specifies exactly what to do (where to explore) next
  - – start at the same point, run 100 times, get the same answer
- While determinism is "comforting", it turns out it is, in general, too restrictive!
  - ➡ as soon as you decide on a starting position, you are relegating many parts of the search space to *never* being explored
- To address this situation, we will look at *nondeterministic*, or *stochastic* algorithms.

Optimisation Algorithms - Roadmap